

Proposed Pull Requests

Fork [stevenrwood/ioSender](#) → upstream [terjeio/ioSender](#) · base branch `master` · 13 branches (PR 8 stacks on PR 3; PRs 9-10 on the integrated keymap/Xbox base; PRs 11-13 off master) · all branches LF-normalized (`.gitattributes * text=auto eol=lf`)

1	RP.Math build fix	<code>pr/fix-rp-math-build</code>	1 file · +6
2	Surface Motor Fault / Warning signals	<code>pr/motor-signals</code>	1 file · +2/-30
3	Console persistence + Key Mappings editor	<code>pr/keymapping_editor</code>	14 files · +1209/-13
4	Connection-loss handling + auto-reconnect	<code>pr/connection-loss-reconnect</code>	9 files · +519/-61
5	Stepper calibration (scratch) tab	<code>pr/stepper-calibration-scratch</code>	5 files · +589
6	Load Folder + Fusion batch-post add-in	<code>pr/load-folder</code>	18 files · +1245/-6
7	High-DPI / large-text layout	<code>pr/highdpi-layout</code>	26 files · +123/-107
8	Xbox controller + Key Mappings editor polish	<code>pr/xbox-mapping</code>	9 files · +1324/-26
9	Configurable main page + flyouts + jog panels (XL)	<code>pr/main-page-layout</code>	29 files · +1385/-33
10	Macro manager + run-only flyout	<code>pr/macro-editor</code>	11 files · +464/-26
11	Macro Processor Enhancements	<code>pr/macro-processor</code>	9 files · +659/-18
12	Onboarding tooltips	<code>pr/onboarding-tooltips</code>	6 files · +83/-24
13	Build documentation (Mega-XL notes + tool-change guide)	<code>pr/build-docs</code>	4 files · +1259

PR 1 Reference RP.Math in CNC Core to fix GCodeEmulator build

Branch pr/fix-rp-math-build

Compare <https://github.com/terjeio/ioSender/compare/master...stevenrwood:ioSender:pr/fix-rp-math-build>

File	+	-
CNC Core/CNC Core/CNC Core.csproj	6	0

CNC Core compiles `GCodeEmulator.cs`, which uses the `RP.Math.Vector3` type, but the project file declared no reference to `RP.Math`. A clean build of CNC Core therefore fails to resolve the type.

This adds the `RP.Math` and `RP.Math.Vector3` references using the *same* `HintPath` convention already used elsewhere in the solution — `CNC Controls.csproj` already references `RP.Math.Vector3` via `..\..\Vector-master\RP.Math.Vector3\bin\Release\`. The fix is six lines and purely additive; no code changes.

PR 2 Surface Motor Fault and Motor Warning signals in Signals display

Branch pr/motor-signals

Compare <https://github.com/terjeio/ioSender/compare/master...stevenrwood:ioSender:pr/motor-signals>

File	+	-
CNC Controls/CNC Controls/SignalsControl.xaml	2	30

Removes the capability-gating Styles that prevented the Motor Fault (**F**) and Motor Warning (**W**) indicators from ever showing.

grblHAL does not advertise these as NEWOPT capability tokens, so the visibility gates were effectively dead code — the LEDs were permanently hidden. Motor Fault / Warning are now always displayed alongside the other signal LEDs (H / S / P), turning red when the corresponding signal triggers. This surfaces the Motor Fault signal as intended, without requiring the firmware to advertise a capability flag.

PR 3 Persist console state and add an in-app Key Mappings editor

Branch pr/keymapping_editor

Compare https://github.com/terjeio/ioSender/compare/master...stevenrwood:ioSender:pr/keymapping_editor

File	+	-
CNC Controls/CNC Controls/AppConfig.cs	14	0
CNC Controls/CNC Controls/AppConfigView.xaml	1	1
CNC Controls/CNC Controls/AppConfigView.xaml.cs	16	3
CNC Controls/CNC Controls/CNC Controls.csproj	7	0
CNC Controls/CNC Controls/ConsoleControl.xaml	3	3
CNC Controls/CNC Controls/ConsoleControl.xaml.cs	14	0
CNC Controls/CNC Controls/ConsoleWindow.xaml.cs	47	0
CNC Controls/CNC Controls/KeyMapEditor.xaml	115	0
CNC Controls/CNC Controls/KeyMapEditor.xaml.cs	606	0
CNC Core/CNC Core/CNC Core.csproj	1	0
CNC Core/CNC Core/KeyPressHandler.cs	111	0
CNC Core/CNC Core/ShortcutKey.cs	136	0
ioSender XL/ioSender XL/MainWindow.xaml.cs	69	3
ioSender/ioSender/MainWindow.xaml.cs	69	3

Persists the console's state across sessions and replaces the old App Settings "Save key mappings" button (which only dumped the current bindings to KeyMap<mode>.xml for hand-editing) with a proper in-app **Edit Key Mappings** dialog. Applies to both **ioSender** and **ioSender XL**.

Console persistence: the three console checkboxes (*Verbose*, *Filter out realtime responses*, *Show all realtime responses*), whether the floating console window is open, and that window's size/position (clamped to the visible desktop so it can never restore off-screen) are all remembered. A new ConsoleShortcut setting (default Esc) toggles the console; it is now a first-class action inside the mappings editor rather than a separate, ad-hoc tab. A shared ShortcutKey helper (parse / storage / display) is used by both main windows, and an AppConfig.ConsoleShortcutChanged event re-registers the key live without a restart.

Key Mappings editor (KeyMapEditor, modal so jog/realtime dispatch can't fire mid-capture):

- One grouped, collapsible outline of every bindable action (grouped by function, like the Load Folder outline; groups remember their expansion per session). Click a row, press a combo to assign; right-click for *Reset to default* / *Clear*.
- Bindings changed from the factory default are highlighted; group headers flag a modified group and offer *Expand all* / *Collapse all* / *Reset group*.
- Per-axis jog keys are folded into the same list; rows and group headers carry tooltips.
- A *No-numpad keys* preset remaps the NumPad jog/step/feed actions to Ctrl+1-8 and [] - = for keyboards without a numeric keypad (Windows exposes no reliable API to detect numpad presence, so this is a user-triggered preset).

The `KeyPressHandler` gains an editor API (`GetActionBindings/GetJogBindings/Apply...`) over its existing function catalog, identifying bindings by reflection method name with a friendly-label map. Persistence still uses the existing `SaveMappings/LoadMappings`.

PR 4 Fix unhandled exceptions on connection loss (e.g. SD card swap)

Branch `pr/connection-loss-reconnect`

Compare <https://github.com/terjeio/ioSender/compare/master...stevenrwood:ioSender:pr/connection-loss-reconnect>

File	+	-
CNC Controls/CNC Controls/AppConfig.cs	5	0
CNC Controls/CNC Controls/SDCardView.xaml.cs	13	0
CNC Core/CNC Core/Comms.cs	74	0
CNC Core/CNC Core/EltimaStream.cs	6	0
CNC Core/CNC Core/Grbl.cs	13	4
CNC Core/CNC Core/GrblViewModel.cs	45	0
CNC Core/CNC Core/SerialStream.cs	179	42
CNC Core/CNC Core/TelnetStream.cs	105	7
CNC Core/CNC Core/WebsocketStream.cs	79	8

When the controller's serial/USB device disappears — for example a board that resets and re-enumerates when its SD card is removed or reinserted — the status-poll timer kept writing to the dead port. The write threw (an `IOException`, then an `InvalidOperationException`: "BaseStream only available when the port is open") on a `System.Timers.Timer` thread, and the unhandled exception terminated the whole process. Opening the SD card tab afterwards threw similarly from `PurgeQueue`.

Changes:

- Guard all stream writes (`IsOpen` + `try/catch`). A device-gone fault now closes the port and starts an auto-reconnect instead of crashing.
- Add a shared, transport-agnostic `Reconnector` (used by serial, network and websocket) exposing `ConnectionLost` / `Reconnected` events. `GrblViewModel` handles them (stop/resume polling, show status); it is wired up at connect time in `AppConfig`.
- **Serial:** refactor open into a reusable `OpenPort()` that probes `GetPortNames()` and re-opens; publish the port only once it is actually open, to avoid a non-null-but-closed race; make `PurgeQueue` tolerant of a closed port; harden the receive handler against a port torn down mid-read.
- **Telnet / websocket:** guarded writes, loss detection (read returning 0 / `OnClose`) and reconnect via the same helper.
- Wrap the poll-timer callback in `try/catch` as a last line of defence.
- Guard the SD card view against acting on a closed connection.

PR 5 Add Stepper calibration (scratch) tab to grbl:Settings

Branch pr/stepper-calibration-scratch

Compare <https://github.com/terjeio/ioSender/compare/master...stevenrwood:ioSender:pr/stepper-calibration-scratch>

File	+	-
CNC Controls/CNC Controls/CNC Controls.csproj	7	0
CNC Controls/CNC Controls/GrblConfigView.xaml	3	0
CNC Controls/CNC Controls/IGrblConfigTab.cs	1	0
CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml	92	0
CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml.cs	486	0

Adds a new tab to grbl:Settings for V-bit *scratch-line* steps/mm calibration over a long span — a more accurate alternative to the short jog-based calibration for the X and Y axes.

It generates an .nc program that scratches paired lines (perpendicular to the calibrated axis) for several candidate steps/mm values. Because GRBL cannot change \$100/\$101 mid-program, each candidate is simulated using the *commanded-distance trick*: the commanded distance $C = \text{span} \times \text{candidate} / \text{current}$, so the machine physically travels the distance that the candidate steps/mm value *would* produce.

The program is loaded into the job view (with an optional save to disk). After cutting, the user measures the actual spacings; the tab then computes and saves the corrected \$100/\$101 from those measurements (an implied value per pair, averaged). The prolog mirrors the Load Folder convention. X/Y only — Z continues to use the existing jog wizard.

Includes a small follow-up commit with high-DPI layout tweaks for the new tab.

PR 6 Add "Load Folder": load a folder of per-toolpath .nc files as one outlined program

Branch pr/load-folder
Compare <https://github.com/terjeio/ioSender/compare/master...stevenwood:ioSender:pr/load-folder>

File	+	-
CNC Controls/CNC Controls/CNC Controls.csproj	2	0
CNC Controls/CNC Controls/FolderPicker.cs	159	0
CNC Controls/CNC Controls/FusionFolderLoader.cs	313	0
CNC Controls/CNC Controls/GCode.cs	109	0
CNC Controls/CNC Controls/GCodeListControl.xaml	37	0
CNC Controls/CNC Controls/GCodeListControl.xaml.cs	84	0
CNC Controls/CNC Controls/JobControl.xaml.cs	4	0
CNC Core/CNC Core/GCodeJob.cs	41	5
CNC Core/CNC Core/GrblViewModel.cs	7	1
Fusion Addin/README.md	95	0
Fusion Addin/install-macos.sh	39	0
Fusion Addin/install-windows.ps1	40	0
Fusion Addin/ioSenderBatchPost/ioSenderBatchPost.manifest	11	0
Fusion Addin/ioSenderBatchPost/ioSenderBatchPost.py	292	0
ioSender XL/ioSender XL/MainWindow.xaml	1	0
ioSender XL/ioSender XL/MainWindow.xaml.cs	5	0
ioSender/ioSender/MainWindow.xaml	1	0
ioSender/ioSender/MainWindow.xaml.cs	5	0

File > Load Folder reads a folder of per-operation files named <seq>_<name>_T<tool>.nc (as produced by the companion Fusion 360 add-in) and assembles them in memory into a single program:

- strips each file's header and program-end footer;
- inserts a G53 G0 Z0 safe-Z retract + M6 T<n> tool change before each toolpath, but only when the file does not already contain an M6 (so any post works);
- optionally restores rapid moves that Fusion Personal Use downgraded to G1;
- shows the result as a collapsible per-toolpath outline (groups default collapsed, and stay expanded while scrolling);
- adds *Start from this toolpath* and *Run just this toolpath* (the latter bounded to that section), each preceded by the G90 G94 / G17 / G21 prolog.

The new FusionFolderLoader is a faithful C# port of the add-in's combine / strip / rapid-restore rules; FolderPicker uses the modern OpenFileDialog (FOS_PICKFOLDERS) for folder selection. GCodeBlock gains a Section tag for outline grouping and an optional AddLineNumbers gate;

GrblViewModel gains IsFolderView and a one-shot RunToBlock end-bound consumed by CycleStart.

Companion Fusion 360 add-in (ioSenderBatchPost): a standalone add-in (no external framework) that posts every operation in every setup to its own <seq>_<name>_T<tool>.nc file in a chosen folder — the input for Load Folder. Combining, tool-change insertion and rapid restoration are intentionally left to ioSender, so the add-in only posts. A post-processor dropdown (default grbl.cps) selects which post Fusion runs per operation. Ships with install-windows.ps1 / install-macos.sh and a README. Fusion auto-discovers it once copied, but it must be enabled once via *Scripts and Add-Ins*.

Note: the Fusion Addin/ folder is an optional companion tool; the maintainer may prefer to host it separately. It can be dropped from the PR if so.

PR 7 Improve high-DPI / large-text-size layout across the UI

Branch pr/highdpi-layout
Compare <https://github.com/terjeio/ioSender/compare/master...stevenrwood:ioSender:pr/highdpi-layout>

File	+	-
CNC Controls Camera/ ... /ConfigControl.xaml	3	3
CNC Controls Lathe/ ... /ConfigControl.xaml	6	6
CNC Controls Probing/ ... /ProbingView.xaml	6	6
CNC Controls Probing/ ... /ProbingView.xaml.cs	3	1
CNC Controls/ ... /BasicConfigControl.xaml	4	4
CNC Controls/ ... /CoordValueSetControl.xaml	6	6
CNC Controls/ ... /FeedControl.xaml	5	5
CNC Controls/ ... /GotoBaseControl.xaml	5	5
CNC Controls/ ... /GotoControl.xaml	1	1
CNC Controls/ ... /JogBaseControl.xaml	2	2
CNC Controls/ ... /LEDControl.xaml	4	5
CNC Controls/ ... /NumericField.xaml	6	3
CNC Controls/ ... /NumericField.xaml.cs	3	1
CNC Controls/ ... /OutlineBaseControl.xaml	8	4
CNC Controls/ ... /OutlineControl.xaml	1	1
CNC Controls/ ... /OutlineFlyout.xaml	1	1
CNC Controls/ ... /OverrideControl.xaml	3	3
CNC Controls/ ... /SpindleControl.xaml	8	8
CNC Controls/ ... /StatusControl.xaml	3	3
CNC Controls/ ... /StepperCalibrationWizard.xaml	2	2
CNC Controls/ ... /StripGCodeConfigControl.xaml	15	9
CNC Controls/ ... /ToolView.xaml	2	2
CNC Controls/ ... /TrinamicView.xaml	1	1
CNC Controls/ ... /WorkParametersControl.xaml	18	18
ioSender XL/ioSender XL/JobView.xaml	6	6
ioSender/ioSender/JobView.xaml	1	1

Makes the UI usable at 125–150% Windows text scaling by replacing fixed control heights/widths with auto-sizing plus `MinWidth/MinHeight`, so labels and fields grow with the font instead of clipping or overlapping. Changes are layout-only — no behavioural changes.

- **Shared controls:** the `NumericField` label column now auto-sizes (`MinWidth = ColonAt`) with a `SharedSizeGroup` so labels align in any container that sets `Grid.IsSharedSizeScope`. `CoordValueSetControl`, `StatusControl`, `LEDControl` (dropped a hard `MaxHeight`) and `OverrideControl` no longer pin `Height/Width`; the **Spindle** group switches its fixed `Width=250` to `MinWidth` so the RPM field is no longer clipped at larger text sizes.

- **grbl:Settings config controls** (Basic / StripGCode / Lathe / Camera) and both Stepper calibration tabs auto-size their rows and the Axis dropdown; the Probing view auto-sizes its left panel with shared-size label alignment.
- **Main window** (JobView XL + ioSender): right-panel grid / columns / panels auto-size instead of fixed 515 / 250 px, and each panel control (WorkParameters, Spindle, Coolant, Outline, Feed, Goto) auto-sizes.

PR 8 Native Xbox controller support (+ Key Mappings editor polish)

Branch pr/xbox-mapping

Depends on PR 3 (pr/keymapping_editor) — this branch is stacked on it; the figures below are the delta on top of PR 3.

Compare https://github.com/stevenwood/ioSender/compare/pr/keymapping_editor...pr/xbox-mapping

File	+	-
CNC Core/CNC Core/XInput.cs	160	0
CNC Core/CNC Core/ControllerService.cs	186	0
CNC Core/CNC Core/ControllerMapper.cs	408	0
CNC Core/CNC Core/CNC Core.csproj	3	0
CNC Core/CNC Core/GrblViewModel.cs	16	0
CNC Core/CNC Core/KeyPressHandler.cs	23	0
CNC Controls/CNC Controls/JogBaseControl.xaml.cs	5	0
CNC Controls/CNC Controls/KeyMapEditor.xaml	166	22
CNC Controls/CNC Controls/KeyMapEditor.xaml.cs	357	4

Adds first-class game-controller support with **zero new dependencies** — the project uses no NuGet, so the controller is read via native **XInput** P/Invoke (xinput1_4.dll, falling back to xinput9_1_0.dll). A ~60 Hz DispatcherTimer poll layer runs in parallel to the keyboard handler.

- **Interop / service:** XInput.cs wraps the gamepad state and vibration structs; ControllerService.cs scans all four slots, edge-detects button down/up, and exposes Connected/Disconnected events, deadzone/normalize helpers and rumble.
- **Button map:** ControllerMapper.cs dispatches a default, fully re-mappable button map to machine actions (A = Cycle Start, B = Feed Hold, X = Spindle Stop, Y = Home, Back = Reset, Start = Unlock, D-pad = step-jog X/Y). Actions route through the same GrblViewModel.ExecuteCommand path as the on-screen buttons and are gated to a live connection in Idle/Jog/Tool states (with a status message when ignored). A short rumble confirms connection.
- **Analog jog:** the left stick proportionally jogs X/Y and the triggers jog Z, via throttled overlapping \$J= moves (jog-cancel on release) so motion stays smooth; speed scales with stick deflection. Enable, left-stick deadzone, max-feed multiple of the Jog panel feed, and per-axis invert are user-configurable. The button map and analog settings persist to ControllerMap.xml.
- **Controller tab** in the Key Mappings editor: live press indicator, per-button action dropdowns (each button's default marked with a leading *), *Restore Defaults* (enabled only when modified), machine-direction-aware tooltips, and the analog-jog settings — machine dispatch is paused while the dialog is open. New JogDistanceProvider/JogFeedProvider hooks let controller jog match the on-screen Jog panel's current step and feed.

The same branch also carries the keyboard-tab polish (changed-binding highlight with per-row/group reset, *Clear* moved into the row menu, column sizing, and the grouped outline now keeps a section expanded while scrolling instead of collapsing it). The branch is fully LF-normalized, so the counts above are the real diff — no line-ending noise.

PR 9 Configurable main page + flyouts + jog panels (ioSender XL)

Branch pr/main-page-layout

Depends on The integrated Key Mappings + Xbox base in master (PRs 3 & 8); the figures below are the delta on top of that base.

Compare <https://github.com/stevenwood/ioSender/compare/master...pr/main-page-layout>

File	+	-
CNC Controls/CNC Controls/AppConfig.cs	37	1
CNC Controls/CNC Controls/AppConfigView.xaml	2	1
CNC Controls/CNC Controls/AppConfigView.xaml.cs	152	0
CNC Controls/CNC Controls/BasicConfigControl.xaml	2	0
CNC Controls/CNC Controls/CNC Controls.csproj	43	0
CNC Controls/CNC Controls/JogBaseControl.xaml	2	2
CNC Controls/CNC Controls/JogBaseControl.xaml.cs	90	2
CNC Controls/CNC Controls/JogSlider.xaml	66	0
CNC Controls/CNC Controls/JogSlider.xaml.cs	51	0
CNC Controls/CNC Controls/KeyboardJoggingControl.xaml	25	0
CNC Controls/CNC Controls/KeyboardJoggingControl.xaml.cs	34	0
CNC Controls/CNC Controls/MainPageEditor.xaml	61	0
CNC Controls/CNC Controls/MainPageEditor.xaml.cs	146	0
CNC Controls/CNC Controls/MainPanelRegistry.cs	100	0
CNC Controls/CNC Controls/NumericField.xaml	11	1
CNC Controls/CNC Controls/NumericField.xaml.cs	38	0
CNC Controls/CNC Controls/OffsetFlyout.xaml	41	0
CNC Controls/CNC Controls/OffsetFlyout.xaml.cs	114	0
CNC Controls/CNC Controls/PanelFlyout.xaml	43	0
CNC Controls/CNC Controls/PanelFlyout.xaml.cs	80	0
CNC Controls/CNC Controls/SidebarItem.cs	23	4
CNC Controls/CNC Controls/UIJoggingControl.xaml	44	0
CNC Controls/CNC Controls/UIJoggingControl.xaml.cs	29	0
CNC Core/CNC Core/KeyPressHandler.cs	22	4
CNC Core/CNC Core/SerialStream.cs	1	1
ioSender XL/ioSender XL/JobView.xaml	8	6
ioSender XL/ioSender XL/JobView.xaml.cs	45	3
ioSender XL/ioSender XL/MainWindow.xaml	3	3
ioSender XL/ioSender XL/MainWindow.xaml.cs	72	5

ioSender XL only. Makes the crowded right-hand panel area configurable instead of fixed. A new **Edit Main Page** dialog (beside *Edit Key Bindings*) is a shuttle editor with three buckets — *Available*, *Main page* (up to six slots), and *Flyouts* — so each named panel can be placed on the main page or as a pinnable right-edge flyout, or left off entirely to reclaim space. Nothing is auto-assigned.

- **Flyouts** share a flush, tab-height border and distribute their content vertically; each has a pin (stays open when another flyout opens, and persists across launches) and a close button.
- **Per-offset flyouts** (G28 / G30 / G54..G59.3 / G92) are tiny *Go* buttons (tooltip shows the live coordinates; G28/G30 also get a *Set* = G28.1/G30.1) so several can be pinned beside the jog panel.
- **Jog panels** — UI jogging (distance + speed) and keyboard default-speed — become assignable panels rendered as compact sliders with a read-only value; Settings:App stays the editor. The jog-arrow radios are hidden only when the UI jog panel is assigned.
- **Settings:App** gains opt-in autosave (with a discard prompt) and a *Reset to default* context-menu entry on numeric fields; the old *Edit Key Mappings* button is renamed **Edit Key Bindings**.

Layout choices persist (panels / flyouts / pins serialized as CSV to sidestep the `XmlSerializer` list-append quirk). Also includes robustness fixes: a resilient `SerialStream.OutCount` that no longer throws on a dropped port, and a deferred main-panel build so startup can't race the config load.

PR 10 Macro manager + run-only flyout

Branch pr/macro-editor

Depends on The integrated base in master (the button sits next to PR 3's "Edit Key Bindings" on Settings:App); the figures below are the delta on top.

Compare <https://github.com/stevenwood/ioSender/compare/master...pr/macro-editor>

File	+	-
CNC Controls/CNC Controls/MacroManagerDialog.xaml.cs	308	0
CNC Controls/CNC Controls/MacroManagerDialog.xaml	81	0
CNC Controls/CNC Controls/MacroExecuteControl.xaml.cs	19	8
CNC Controls/CNC Controls/AppConfig.cs	19	0
CNC Controls/CNC Controls/AppConfigView.xaml.cs	10	0
CNC Controls/CNC Controls/MacroExecuteControl.xaml	8	16
CNC Controls/CNC Controls/CNC Controls.csproj	7	0
CNC Core/CNC Core/GCode.cs	5	0
CNC Controls/CNC Controls/JobControl.xaml.cs	3	2
CNC Controls/CNC Controls/MacroToolbarControl.xaml.cs	3	0
CNC Controls/CNC Controls/AppConfigView.xaml	1	0

Adds an **Edit Macros** button to the Settings:App page opening a macro manager — a DataGrid with one row per macro, in the spirit of Fusion's *Scripts and Add-Ins* dialog — and reworks the on-screen macro flyout for running.

- **Name** and a **Prompt** (confirm-before-run) flag are edited in-line; an assignable **Key** column (F1–F12) sets the function key (keys are unique; *Create* auto-assigns the first free one); a **File** column shows the referenced file for @<path> macros (full path on hover), blank for inline.
- **Edit** edits the macro's definition (inline G-code, or the @<path> reference line) and reads it back; **View** opens what it points to — the referenced file for an @<path> macro (created if missing, to author a new one), otherwise the code. **Create/Delete** add and remove rows.
- The **macro flyout** is now run-only: a macro-name dropdown plus a **Run** button (the Edit button is gone — editing lives in the manager). It pre-selects the most recently run macro, persisted in `Config.LastMacroId` and updated from every run path (flyout, toolbar, F-keys), falling back to the first.

The macro's function key is decoupled from its Id via a new `Macro.FKey` field (legacy configs migrated on load: Id n keeps Fn). The @<path> *run-time* resolution — loading and running the referenced file, with directive processing — is provided by **PR 11**; this PR adds the manager/flyout support for authoring, viewing, and repointing references.

PR 11 Macro Processor Enhancements

Branch `pr/macro-processor`

Depends on Off master, independent of PR 10. The two are complementary: PR 10's manager *authors* macros, this *executes* them.

Compare <https://github.com/stevenwood/ioSender/compare/master...pr/macro-processor>

File	+	-
CNC Controls/CNC Controls/MacroProcessor.cs	582	0
CNC Controls/CNC Controls/MacroToolBarControl.xaml.cs	1	1
CNC Controls/CNC Controls/MacroExecuteControl.xaml.cs	1	1
CNC Controls/CNC Controls/JobControl.xaml.cs	1	2
CNC Controls/CNC Controls/CNC Controls.csproj	1	0
CNC Core/CNC Core/GCodeParser.cs	31	12
CNC Core/CNC Core/NGCExpr.cs	22	0
CNC Core/CNC Core/Grbl.cs	11	0
CNC Core/CNC Core/GrblViewModel.cs	9	2

A sender-side **macro pre-processor** (`MacroProcessor.Run`) that interprets parenthesised directives before the remaining lines are streamed to the controller. All macro entry points — the top toolbar, the on-screen macro flyout, and the F-key handler — now route through it (previously the toolbar called `ExecuteMacro` directly and silently ignored every directive).

Directives, each a no-op the controller would treat as a comment, interpreted by the sender instead:

- **(PREREQ, cond, ...)** — require machine state *before* anything streams; if unmet the macro aborts with a message and nothing is sent. Conditions: `homed`, `idle`, `tlo`, `noalarm`, `connected`; stored positions / work offsets `g28`, `g30`, `g92`, `g54`–`g59.3` ("set" if any axis offset is non-zero, read fresh from `$#`); and any other token is required to be a controller `$I` build option (NEWOPT, e.g. `EXPR`), matched exactly and case-sensitively. `homed` reads `grblHAL's live sys.homed.mask` from `$#` rather than the change-based status `H: cache`, which can stay stale after a position-loss alarm.
- **(MBOX, [buttons,] message)** — modal hold; OK/OKCANCEL/YESNO, Cancel/No aborts the rest.
- **(WAITIDLE)** — hold streaming until the controller finishes what was sent and returns to Idle — needed after a controller-side job such as `$F=<file>`, which acks immediately and drops sender input while it runs.
- **(PROMPT param, default [, label])** — collect input values in a single up-front dialog and bind them two ways: assign the globals on the controller (so `$F=` files can read them) and substitute the references in the streamed body.
- **@<path>** — a macro whose body is a single `@<path>` line is a reference to an external file: that file's current contents are loaded and run in its place, re-read on every run — so a macro can be developed by editing the file directly. The loaded contents go through the same directive pipeline. (PR 10 adds the manager/flyout support for authoring and repointing these references.)

Supporting core changes so macros stream cleanly to an NGC-capable controller, all in the spirit of "don't reject what the controller can evaluate": `ExecuteMacro` now inherits the controller's expression

support (as file streaming already did) and forwards verbatim any line its own parser cannot handle; `GCodeParser` recognises **named O-word subroutine calls** (`O<name> CALL [...]`) and forwards them to the controller, which resolves the named sub from its filesystem; and the `$# [HOME:...:mask]` homed-axes mask is parsed for the authoritative homed check.

PR 12 Onboarding tooltips

Branch	pr/onboarding-tooltips
Depends on	Off master, independent. Aimed at users new to CNC.
Compare	https://github.com/stevenwood/ioSender/compare/master...pr/onboarding-tooltips

File	+	-
CNC Controls/CNC Controls/Converters.cs	50	0
CNC Controls/CNC Controls/StatusControl.xaml	3	0
CNC Controls/CNC Controls/StripGCodeConfigControl.xaml	10	5
CNC Controls/CNC Controls/JogUiConfigControl.xaml	10	9
CNC Controls/CNC Controls/JogConfigControl.xaml	7	7
CNC Controls/CNC Controls/BasicConfigControl.xaml	3	3

Tooltips aimed at users new to CNC, where ioSender's UI is otherwise terse.

- **State field** — a context-aware tooltip explains the current controller state in plain language (Idle, Run, Hold, Check, Tool, Door, ...). For an **alarm** it appends the controller's specific message — e.g. ALARM 4: Probe fail ..., pulled from the GrblAlarms enumeration loaded at connect — plus how to recover, so a bare ALARM:4 is self-explanatory instead of a code to look up. (A new GrblStateToTooltipConverter drives a ToolTip binding on the State box.)
- **Settings:App** — a “what it does and why it’s useful” tooltip on each application setting (the grbl settings page already self-documents in its right-hand pane, so it’s left alone). Keyboard jogging (fast/slow/step feed & distance, link-step-to-UI), UI jogging (the four distance and four speed presets, noting distance and speed are selected *independently*), and GCode command stripping (M6/M7/M8/G61-G64, for machines lacking the matching hardware). Also fixes the “Keep MDI focus” tooltip (it had been copy-pasted from the buffering option) and the circular “Send comments” one.

PR 13 Build documentation: Mega-XL upgrade notes + repeatable tool-change guide

Branch pr/build-docs
Depends on Off master, independent. Documentation only — no code or build changes.
Compare <https://github.com/stevenwood/ioSender/compare/master...pr/build-docs>

File	+	-
docs/Mega-XL-Upgrades.html	840	0
docs/Mega-XL-Upgrades.pdf	bin	0
docs/Repeatable-Tool-Change.html	419	0
docs/Repeatable-Tool-Change.pdf	bin	0

Two self-contained HTML documents (each rendered to a companion PDF), added under a new docs/ folder. Purely additive; touches no source.

- **Repeatable-Tool-Change** — a turnkey, novice-friendly tool-change guide built around a **G59.3 fixed toolsetter** and a T<n>/M6 flow, with a top-down machine travel diagram (color-coded paths). Aimed at the goal of repeatable, hands-off tool changes for users new to CNC.
- **Mega-XL-Upgrades** — build notes for an upgraded Kickstarter v1.0 Mega V XL: frame/spindle/motion/probing/power sections, a back-of-enclosure two-panel harness model (hand-built SVG diagrams), a cable & wiring schedule, and an **interactive bill-of-materials worksheet** — an in-page editor (double-click to edit Qty / Unit / Description / Source, live extended & category subtotals & grand total, URL shortening, Save-to-download) plus a linked table of contents.

The Mega-XL notes are specific to one machine and may be of limited upstream interest — they can be hosted separately or dropped from the PR. The repeatable tool-change guide is more generally applicable. The associated controller macros (`tc.macro`, `probe_tfl.macro`) and the ATC provisioning UI are intentionally *not* in this PR — they remain with the macro work, pending firmware support.